

UNIT 3

MEMORY MANAGEMENT

Syllabus:

Memory Management: Introduction, Logical Versus Physical Address Space, Dynamic Loading, Dynamic Linking, Swapping, Contiguous Allocation, Partitioned Memory Allocation, Paging, Virtual Memory Management-Segmentation, Segmentation with Paging.

File System: File Concepts, File Attributes, File Operations, File Types, File Structure, Access Methods, Directory structure, File-System Structure, Allocation Methods- Contiguous Allocation, Linked Allocation and Indexed Allocation, Free-Space Management.

MEMORY MANAGEMENT:

Memory management is allocating, freeing, and re-organizing memory in a computer system to optimize the available memory or to make more memory available. It keeps track of every memory location(if its is free or occupied).

Memory management is a technique of controlling and managing the functionality of Random-access memory (primary memory). It is used for achieving better concurrency, system performance, and memory utilization.

LOGICAL AND PHYSICAL ADDRESS SPACE

LOGICAL ADDRESS

- The address **generated by the CPU** during a program's execution.
- It's what a **program thinks** is the memory location.
- Also called **virtual address**.
- Example: Program says “store data at address 1000”.

PHYSICAL ADDRESS

- The **actual location in RAM (main memory)** where the data is stored.
- The **Memory Management Unit (MMU)** maps the logical address to a physical address.
- Example: The real memory location might be address 5000.

Parameter	LOGICAL ADDRESS	PHYSICAL ADDRESS
Basic	generated by CPU	location in a memory unit
Address Space	Logical Address Space is set of all logical addresses generated by CPU in reference to a program.	Physical Address is set of all physical addresses mapped to the corresponding logical addresses.
Visibility	User can view the logical address of a program.	User can never view physical address of program.

Generation	generated by the CPU	Computed by MMU
Access	The user can use the logical address to access the physical address.	The user can indirectly access physical address but not directly.
Editable	Logical address can be change.	Physical address will not change.
Also called	virtual address.	real address.

LOADING

The process of **bringing a program from the hard disk (secondary storage) into the main memory (RAM)** so that the CPU can execute it.

LINKING

Linking is the process of **combining different parts of a program** (like object files and libraries) into a single executable file so it can run.

DYNAMIC LOADING

Dynamic Loading **is when a program** loads a part of its code (like a function or module) into memory only when it is needed during execution.

1. It loads code **on demand**, not all at once.
2. Saves **memory**, because unused code is not loaded.
3. Happens **during program execution (runtime)**.
4. Requires programmer to use **special system calls**
5. Useful for **large programs**.

Example: In a photo editing app, the "Filters" feature is loaded **only when you click on it**—not when the app starts.

DYNAMIC LINKING

Dynamic Linking is the process of **linking external libraries** to a program **during runtime**, not at compile time. This means the program doesn't carry all the code; it uses functions from shared libraries only when needed.

- It links **external libraries or modules** to the program **while it's running**.
- Saves memory, as the program **doesn't carry all library code**.
- Reduces program size since libraries are **not included in the executable**.
- Managed by the **operating system** and **runtime linker**.
- Example: A program calls a function from `math.h`, and the system links the math library at **runtime**.

Example: when you open a web browser like **Google Chrome**. It links to specific libraries (e.g., for playing videos or viewing PDFs) only when needed, instead of loading them all at once, which saves memory and improves performance.

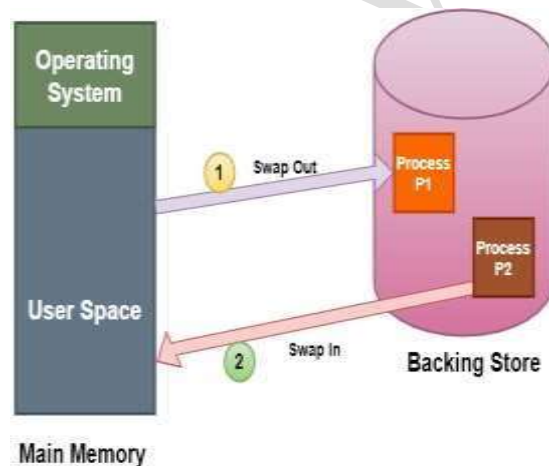
	Dynamic Loading	Dynamic Linking
What it does	Loads parts of the program (like functions) into memory only when needed.	Links external libraries or modules to the program during execution.
When it happens	During program runtime, as parts are needed.	During program runtime, when a function from an external library is called.
Memory Usage	Saves memory by loading only the required parts.	Saves memory by linking to shared libraries without including them in the program.
Control	Managed by the program (programmer handles loading).	Managed by the operating system or runtime linker.
Example	Loading a function in a photo editing app only when the user applies a filter.	Linking a video player to a codec library when playing a video.

SWAPPING:

Swapping is a memory management technique that temporarily swaps processes from main memory to secondary memory or vice versa.

There are two steps in the process of Swapping in the operating system:

1. Swap In: Swap-in is the process of bringing a process from secondary storage to main memory.
2. Swap Out: Swap-out takes the process out of the Main memory and puts it in the secondary storage.



In the above diagram, process P1 is swap out so the process with more memory requirement or higher priority will be executed to increase the overall efficiency of the operating system. While the process P2 is swap in for its execution from the secondary memory to the main memory(RAM).

Advantages of Swapping

1. It helps the CPU to manage multiple processes within a single main memory.
2. It helps to create and use virtual memory.
3. Swapping allows the CPU to perform multiple tasks simultaneously. Therefore, processes do not have to wait very long before they are executed.
4. It improves the main memory utilization.

Disadvantages of Swapping

1. If the computer system loses power, the user may lose all information related to the program in case of substantial swapping activity.
2. If the swapping algorithm is not good, the composite method can increase the number of Page Fault and decrease the overall processing performance.

CONTIGUOUS ALLOCATION

Contiguous Allocation is a method of storing files where each file is stored in a **single contiguous block of memory**.

Working:

- **Process Loading:** When a process is loaded, it's assigned a **single contiguous block** in memory.
- **Address Translation:** The program uses logical addresses, which are converted to **physical addresses** using this formula:
Physical Address = Base Address + Offset
- **Memory Management:** The operating system keeps track of which memory blocks are free and which are in use.

Imagine a system with **100 MB** of memory and 3 processes:

- **P1** needs 20 MB.
- **P2** needs 30 MB.
- **P3** needs 40 MB.

The memory would be allocated as follows:

- **P1:** 0 to 20 MB (20 MB)
- **P2:** 20 to 50 MB (30 MB)
- **P3:** 50 to 90 MB (40 MB)
- **Free:** 90 to 100 MB (10 MB)

If **P2** terminates, the space becomes free, but it may not be possible to allocate a new process that needs 50 MB due to the free space being **non-contiguous** (split into smaller parts). This is called **external fragmentation**.

Advantages of Contiguous Allocation:

1. **Simple to Implement:** Easy to manage, especially in systems with small memory needs.
2. **Fast Access:** Memory access is quick because everything is stored together in one block.
3. **Great for Sequential Data:** Works well for programs that need to access data in order, one piece after another.

Disadvantages of Contiguous Allocation:

1. **External Fragmentation:** Over time, free space gets scattered, making it hard to allocate large programs even if there's enough total free memory.
2. **Internal Fragmentation:** Fixed partitions can lead to wasted memory within allocated blocks.
3. **Inflexibility:** A process must fit entirely within one block, limiting efficient use of memory.

PARTITIONED MEMORY ALLOCATION

Partitioned Memory Allocation is a method of dividing memory into **fixed-size or variable-size partitions** to store processes. Each partition can hold one process, and processes are allocated memory from one of these partitions.

How It Works:

1. **Memory Division:** The operating system divides the total memory into **partitions** (either fixed-size or variable-size).
2. **Process Allocation:** Each process is loaded into one of the partitions, based on its size.
3. **Fixed or Variable Size:**
 - In **fixed partitioning**, the memory is divided into **equal-sized** partitions.
 - In **variable partitioning**, memory is divided into **partitions of varying sizes** based on the process's needs.

There are two main types of **Partitioned Memory Allocation**:

1. **Fixed Partitioning:**
 - Memory is divided into **equal-sized partitions**.
 - Each process is allocated a partition, and if a process is smaller than the partition, the remaining space is **wasted** (internal fragmentation).
 - The **number of partitions** is fixed and cannot change during runtime.

Example: If you have 100 MB of memory and create 5 partitions of 20 MB each, each process gets a 20 MB partition, even if it needs less.

2. **Variable Partitioning:**
 - Memory is divided into **partitions of varying sizes** based on the size of the process.
 - The operating system dynamically allocates partitions to processes based on their needs.
 - This minimizes **internal fragmentation**, but can lead to **external fragmentation** (unused spaces scattered throughout memory).

Example: If a process requires 30 MB, it is allocated a 30 MB partition. If another process requires 50 MB, it gets a 50 MB partition, leaving the rest of the memory available for future use.

Advantages:

1. **Simple to implement.**
2. **Efficient** for processes that fit neatly into partitions.

Disadvantages:

1. **Internal Fragmentation** (unused space within a partition).
2. **Wasted space** if processes don't exactly fit into partitions.

PAGING

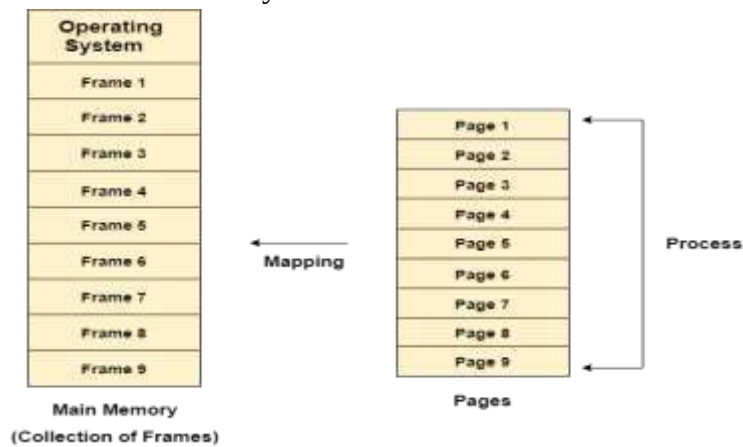
Paging is a memory management technique where both the process and memory are divided into small, fixed-size blocks called **pages** and **frames**, respectively.

This allows the operating system to store parts of a process in different locations in the main memory.

How It Works:

1. **Process Division:** A process is divided into **pages**.
2. **Memory Division:** The main memory is divided into **frames** (same size as pages).

3. **Storing Pages:** Each page of a process is loaded into **any frame** of the memory. The pages do not need to be contiguous in memory, so they can be stored in different places.
4. **Page Table:** The operating system maintains a **page table** that maps each page of the process to a specific frame in memory.



Example

- **Main memory size:** 16 KB
- **Frame size:** 1 KB (so 16 frames)
- **Process size:** 4 KB per process
- **Page size:** 1 KB (matching frame size)

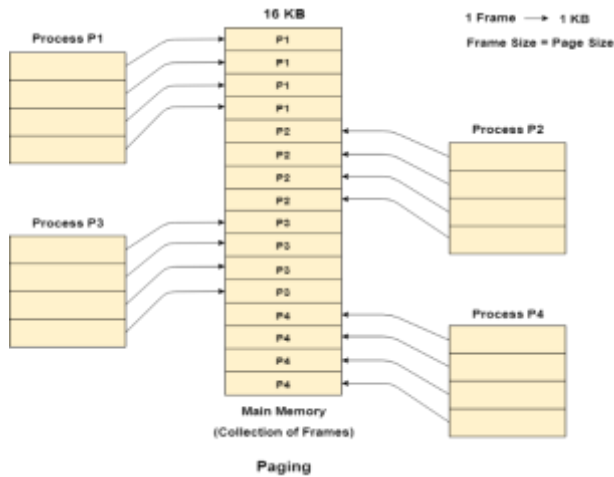
So, for **4 processes** (P1, P2, P3, P4), each process is divided into 4 pages (1 KB each). The pages can be loaded into any available frames in the memory, not necessarily in order.

For example:

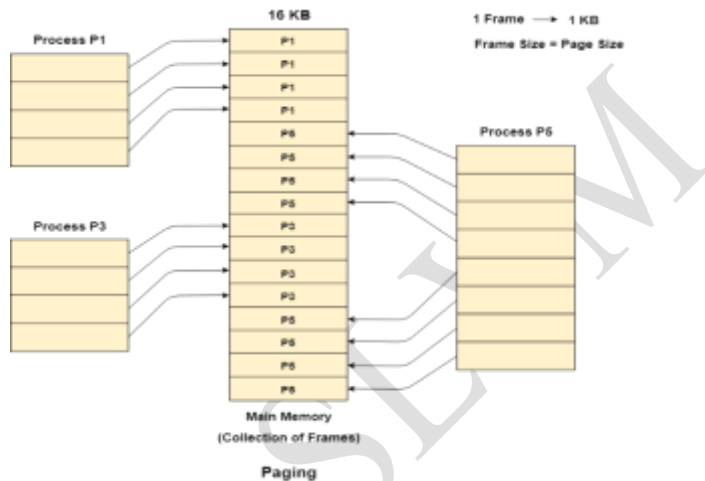
- **P1's page 1** could go into **frame 1**.
- **P2's page 1** could go into **frame 3**.
- **P3's page 1** could go into **frame 5**.
- **P4's page 1** could go into **frame 7**.

If **P5** arrives and there are available frames (e.g., **P2's frame** and **P4's frame** are free), it can load its pages into those frames, even though they aren't contiguous.

Initially, all the frames are empty therefore pages of the processes will get stored in the contiguous way. Frames, pages and the mapping between the two is shown in the image below.



Given the fact that, we have 8 non contiguous frames available in the memory and paging provides the flexibility of storing the process at the different places. Therefore, we can load the pages of process P5 in the place of P2 and P4.



Advantages:

- **No need for contiguous memory:** Pages can be loaded into any free frame.
- **Flexible memory management:** Helps avoid external fragmentation.

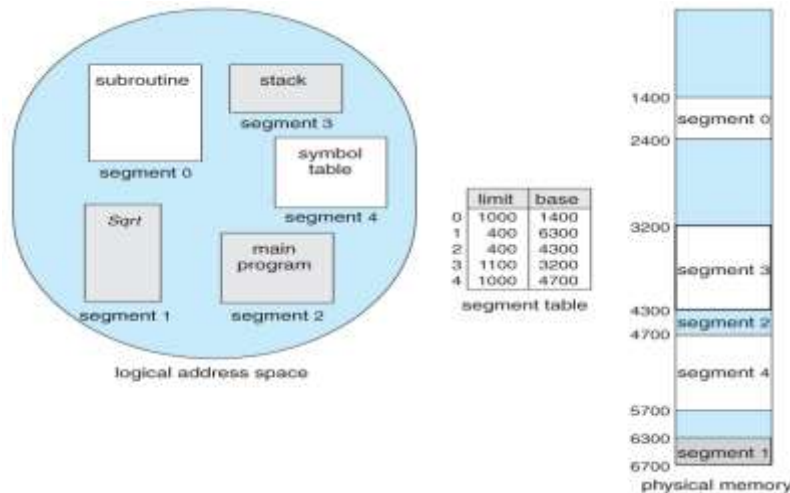
This technique makes memory usage more efficient and allows processes to be loaded into different parts of memory as needed.

VIRTUAL MEMORY MANAGEMENT

Virtual memory is a memory management technique that gives the illusion to programs that they have access to a larger amount of memory than is physically available in the computer's RAM.

SEGMENTATION

Segmentation is a memory management technique in which a process is divided into logical segments based on the program's structure, such as code, data, stack, and heap. This division allows the operating system to treat different parts of a program separately and map them independently to physical memory.



Key Concepts:

- **Logical Address Space:** The program's address space, which is divided into logical segments. Each segment is a contiguous block of memory that represents a specific part of the program.
- **Physical Memory:** The actual physical memory (RAM) where the program is stored during execution.
- **Segment Table:** A table used by the operating system to map each segment's base address (starting address in memory) and limit (size of the segment).

How Segmentation Works

1) Dividing the Program into Segments

The program is split into parts:

Code Segment – has the program instructions.

Data Segment – stores global variables.

Stack Segment – stores function calls and local variables.

Heap Segment – for memory created at runtime (like using malloc in C).

2) segment Table

The OS uses a **segment table** to manage memory.

For each segment, it stores:

Base – where it starts in memory.

Limit – how big it is.

3) Accessing Memory

Programs don't directly use physical memory.

They use a **logical address**:

Segment Number (e.g., 0 for code)

Offset (e.g., byte 10 inside the segment)

The OS uses the segment table to convert it into a **physical address**.

Example

Segment 0 (Code): Starts at address 1000, size 400 → Uses memory 1000 to 1399.

Segment 1 (Data): Starts at 3200.

If the program wants byte 20 of the data segment:

Base = 3200

Offset = 20

$$\text{Physical Address} = 3200 + 20 = 3220$$

Advantages

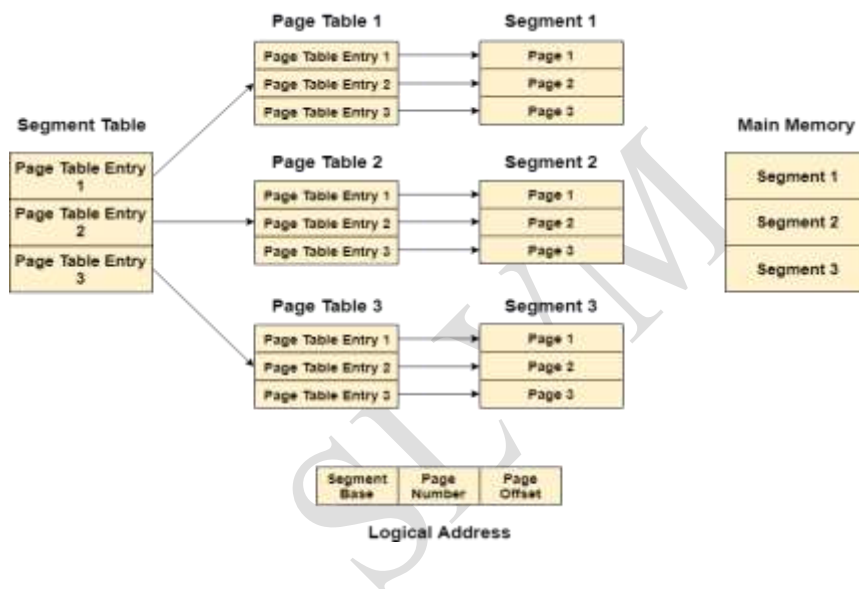
- Easy to manage program parts (code, data, etc.)
- Each part can grow independently.

Disadvantages

- Free memory can become scattered (**external fragmentation**).
- More work for the OS to manage.

SEGMENTATION WITH PAGING

Segmentation with Paging is a memory management technique where each program is divided into segments, and each segment is further divided into fixed-size pages. This combines the logical organization of segmentation with the efficient memory use of paging.



Key Concepts:

- **Segmentation** divides memory into logical parts (e.g., code, data).
- **Paging** breaks each segment into fixed-size pages.
- A **segment table** maps segments to page tables, and a **page table** maps pages to physical memory.
- The **offset** within the page gives the exact memory location.

How Segmentation with Paging Works:

1. The program's memory is divided into **segments** (e.g., code, data).
2. Each segment is divided into **pages** of fixed size.
3. The program gives an address with:
 - **Segment number** (which part of the program),
 - **Page number** (which page in that segment),
 - **Offset** (exact location within the page).
4. The system looks up the **segment table** to find the page table, then looks up the **page table** to find the frame in physical memory.

5. It adds the **offset** to the frame's base address to get the final physical address.

This method combines the benefits of logical structure (segmentation) and efficient memory use (paging).

Advantages of Segmentation with Paging

- **No External Fragmentation:** Fixed-size pages mean free space isn't wasted between segments.
- **Good Memory Use:** You get the clear organization of segments and the space efficiency of paging.
- **Easy Protection/Sharing:** You can set permissions or share individual segments or pages.

Disadvantages of Segmentation with Paging

- **More Complex:** You need both segment tables and page tables, which makes the system harder to manage.
- **Higher Overhead:** Keeping track of extra tables and entries uses more memory and CPU time.

FILE SYSTEM

A file is a collection of data or information stored on a computer or storage device, such as a hard disk, SSD, or USB drive.

Files can contain various types of data, such as text, images, videos, audio, or programs, and are identified by a name, often with an extension that indicates the type of data within the file.

FILE CONCEPT

- Computers can store information on various storage media.
- Operating system provides a uniform logical view of information storage. This logical storage unit is called as a file.
- Files are mapped by operating system onto physical devices.
- File is a named collection of related information that is recorded on secondary storage.
- Files represent both the program and the data. Data can be numeric, alphanumeric, alphabetic or binary.
- Many different types of information like source programs, object programs, executable programs, numeric data, payroll recorder, graphic images, and sound recordings and so on can be stored on a file.

FILE ATTRIBUTES

- **Name:** The symbolic file name is the only information kept in human readable form.
- **Identifier:** The unique tag, usually a number, identifies the file within the file system. It is the nonreadable name for a file.
- **Type:** This information is needed for systems that support different types.
- **Location:** This information is a pointer to a device and to the location of the file on that device.
- **Size:** The current size of the file and possibly the maximum allowed size are included in this attribute.
- **Protection:** Access control information determines who can do reading, writing, execute and so on.
- **Time, data and User Identification:** This information must be kept for creation, last modification and last use. These data are useful for protection, security and usage monitoring.

FILE OPERATIONS

- **Create:** Creates a new file in the file system.
- **Open:** Opens an existing file for reading, writing, or both.

- **Read:** Reads data from an open file
- **Write:** Writes data into a file.
- **Close:** Closes an open file and releases resources.
- **Delete:** Removes a file from the storage device.
- **Rename:** Changes the name of an existing file.
- **Copy:** Creates a duplicate of a file.
- **Move:** Moves a file to another location.
- **Permissions:** Changes the file's read, write, or execute permissions.

TYPES OF FILE SYSTEM

A file system is a method an operating system uses to store, organize, and manage files and directories on a storage device.

1. **Text Files:** Contain plain text and can be opened with a text editor.
Example: .txt, .csv, .md
2. **Binary Files:** Store data in binary format, not readable by humans.
Example: .jpg, .png, .exe
3. **Executable Files:** Contain programs that can be executed by the operating system. Example: .exe (Windows), .bin (Linux)
4. **Image Files:** Store graphical data or images. Example: .jpg, .png, .gif
5. **Audio Files:** Contain sound or music. Example: .mp3, .wav, .aac
6. **Video Files:** Store moving images or videos. Example: .mp4, .avi, .mov
7. **Database Files:** Contain structured data and are used by database management systems. Example: .db, .sql
8. **Document Files:** Contain formatted text and may include multimedia.
Example: .pdf, .docx, .pptx
9. **Archive Files:** Store compressed or packaged files. Example: .zip, .tar, .rar
10. **System Files:** Files required by the operating system to function properly.
Example: .dll, .sys, .bin

FILE STRUCTURE

The **file structure** refers to the organization and layout of data within a file. It determines how data is stored, accessed, and managed in the file system.

File Structure Components:

1. **File Header:** Contains metadata like file name, type, size, and permissions.
2. **File Body:** Contains the actual data stored in the file.
3. **File Footer:** Some file systems include a footer for error checking or data

File structure involves both the logical structure (how users interact with files) and the physical structure (how files are stored on disk).

1. Logical File Structure

This refers to the way files are organized and accessed by users. It is independent of the physical storage and typically involves a hierarchy or logical arrangement of files.

- **File Name:** A unique identifier for the file, usually with an extension (e.g., .txt, .jpg).

- **File Directory:** A container that holds information about files and other directories. It helps in organizing files in a hierarchical way (folders and subfolders).
- **File Path:** Specifies the location of a file within the directory structure, either **absolute** or **relative**.
- **File Metadata:** Includes information like file size, creation date, last modified date, and permissions.

2. Physical File Structure

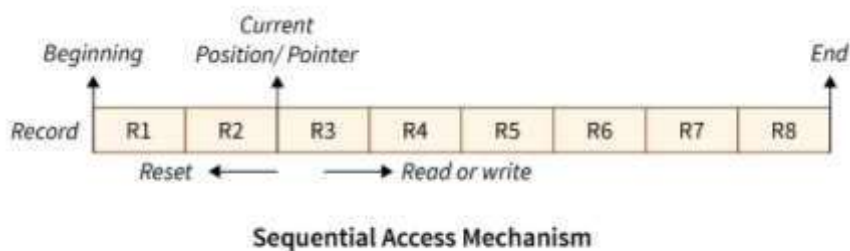
This refers to how the file's data is stored physically on storage devices (e.g., hard disks, SSDs). It is the underlying implementation that supports the logical structure.

- **File Blocks:** Files are broken into small **blocks** (or clusters) of data, which are stored on disk. Each block has a unique address.
- **Inode:** A data structure that stores metadata about the file, such as file size, location on disk, and permissions. It doesn't store the file's name.
- **Disk Allocation:** The method by which blocks are allocated to store files.

FILE ACCESS METHODS:

1) Sequential Access:

- Sequential access is based on the tape model of a file.
- Information in the file is processed in order, one record after the other.
- A read operation reads the next portion of the file and automatically advances the file pointer.
- A write operation appends to the end of the file and the file pointer.

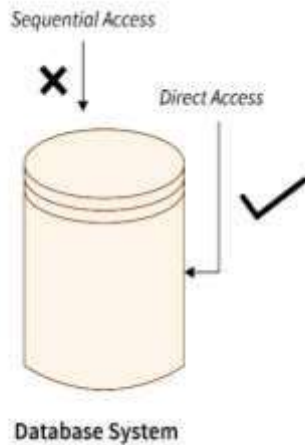


- Data is accessed one record right after another record in an order.
- When we use read command, it move ahead pointer by one
- When we use write command, it will allocate memory and move the pointer to the end of the file.

2) Direct Access

- Direct access method also known as relative access method.
- A Direct/Relative file access mechanism is mostly required with the database systems.

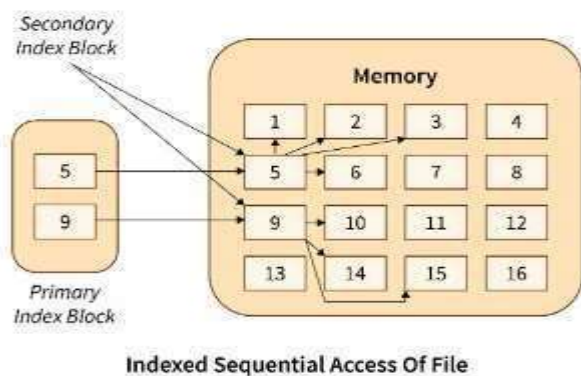
Example: Assume that each block of storage holds four records and that the record we want to access is stored in the tenth block. In such a situation, sequential access will not be used since it will have to traverse all of the blocks to get to the required record, while direct access will allow us to access the required record instantly.



- The files can be immediately accessed decreasing the average access time.
- In the direct access method, in order to access a block, there is no need of traversing all the blocks present before it.

3) Index sequential method

- This method is built on the top of direct access method.
- It involves the construction of an index for a file.
- The index contains pointers to the various blocks.
- To find an entry in the file, the index is searched first and the pointer is then used to access the file directly to find the desired entry.

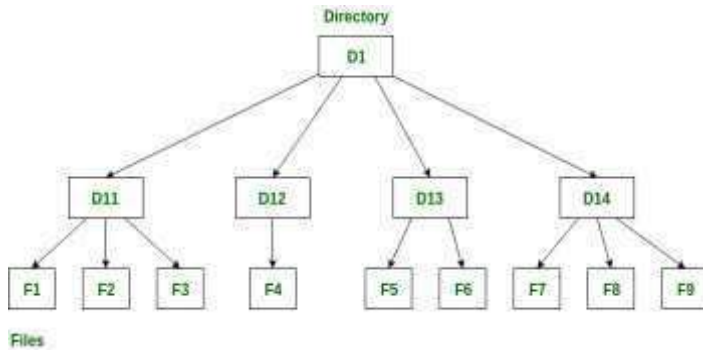


- This mechanism is built up on base of sequential access.
- An index is created for each file which contains pointers to various blocks.
- Index is searched sequentially and its pointer is used to access the file directly.

DIRECTORY STRUCTURE:

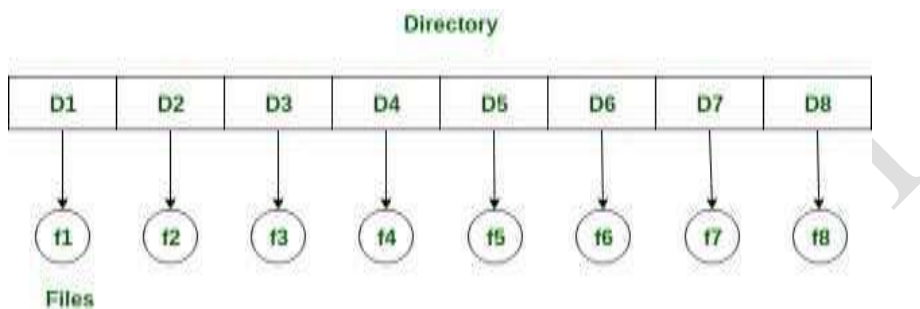
Directories maintain information about files.

- A directory is a container that is used to contain folders and files.
- It organizes files and folders in a hierarchical manner.
- The files systems can be very large. Some systems stores millions of files on the disk.
- To manage all this data we need to organize them.



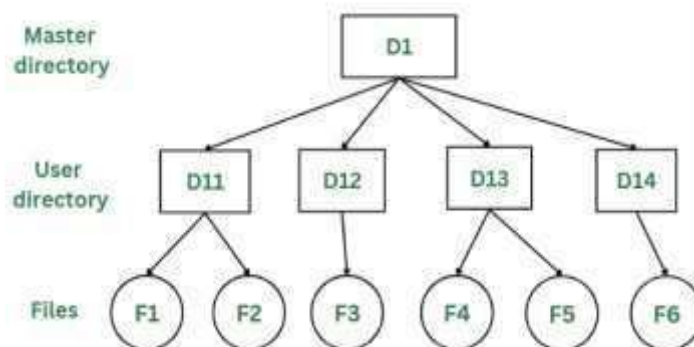
1) Single-level directory

- In a single-level directory system, all the files are placed in one directory.
- It has significant limitations when the no. of files or when there is more than one user. Since all the files are in same folder, they must have unique name.
- In it, all files are contained in the same directory which makes it easy to support and understand.



2) Two-level directory

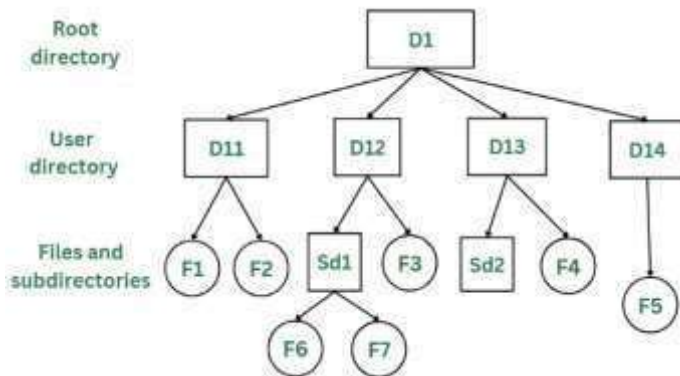
- A single level directory often leads to confusion of files names among different users.
- The solution to this problem is to create a separate directory for each user.
- In the two-level directory structure, each user has their own user files directory (UFD).
- This master block contains the addresses of the directories of the users. The problem with this structure is that it effectively isolates one user from another.



3) Tree Structure/ Hierarchical Structure:

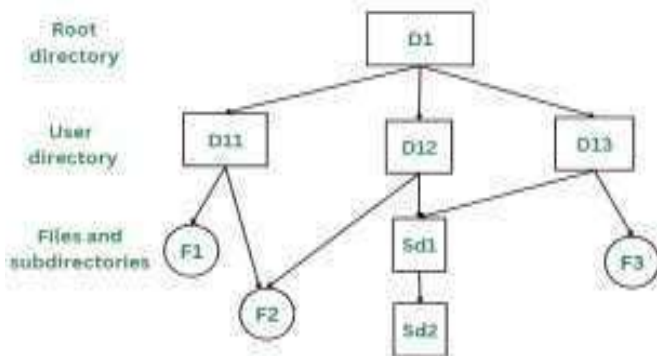
- User can create files and subdirectories too, which was a disadvantage in the previous directory structures.
- This directory structure resembles a real tree upside down, where the root directory is at the peak.
- This root contains all the directories for each user.
- The users can create subdirectories and even store files in their directory.

- A user do not have access to the root directory data and cannot modify it. And, even in this directory the user do not have access to other user's directories.



4) Acyclic Graph Structure:

- It is an extension of the tree-structured directory structure.
- In the tree structured directory, files and directories starting from some fixed directory are owned by one particular user.
- In the acyclic structure, this prohibition is taken out and thus a directory or file under directory can be owned by several users.



FILE SYSTEM STRUCTURE

1. Application Programs:

- The topmost layer consists of application programs (like text editors, web browsers, etc.) that interact with the file system to store, retrieve, and manipulate data.
- Applications make use of the file system through system calls to access or modify files.

2. Logical File System:

- This layer deals with the **logical** organization of files (like filenames, directories, and file access methods).
- It abstracts the physical structure of the storage and provides a way for applications to interact with files using file names, directory structures, and file permissions.

3. File-Organization Module:

- This module handles the **file organization** on disk (such as file allocation, indexing, and block management).

- It defines how files are stored on the storage medium and how file data is accessed. For example, it handles operations like sequential or indexed allocation.

4. Basic File System: • The basic file system manages the actual data storage on the device. It translates logical operations (from the logical file system) into physical operations (such as disk reads and writes).

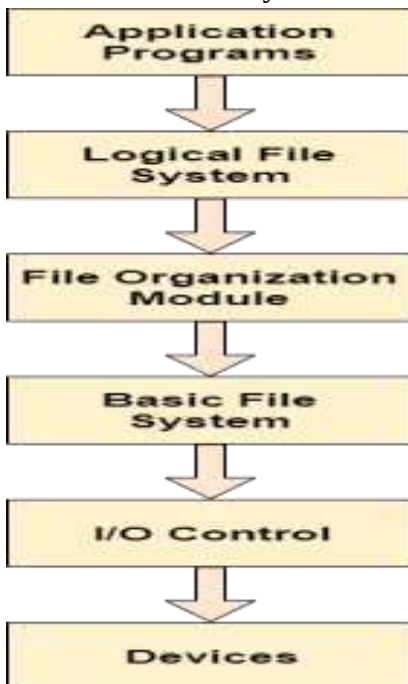
- It is responsible for performing operations like reading data blocks and writing data blocks on the physical storage.

5. I/O Control:

- This layer is responsible for managing the **input/output operations** between the file system and the physical storage devices (like hard drives or SSDs).
- It contains drivers and modules that manage communication with the hardware and control data transfers.

6. Devices:

- The lowest level in the file system hierarchy consists of the **physical devices** like hard drives, SSDs, or other storage media.
- This is where the actual data is stored and retrieved. The devices are managed by the operating system's I/O control system.



Flow Explanation:

- When an **application program** needs to access or store data, it sends a request to the **logical file system**, which abstracts the file's name and organization.
- The **file-organization module** translates these requests into a storage scheme (like indexing or allocation on disk).
- The **basic file system** manages the disk blocks that store the actual data.
- The **I/O control** sends low-level commands to the hardware, instructing it to perform read or write operations.
- The **devices** perform the actual physical operations (reading/writing data).

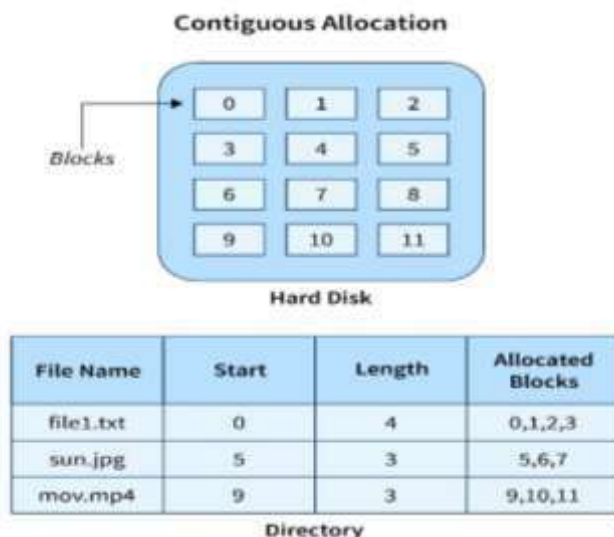
ALLOCATION METHODS

The File allocation methods in OS are different ways that are used for storing the file on the hard disk.

1) Contiguous Allocation

In contiguous file allocation, the block is allocated in such a manner that all the allocated blocks in the hard disk are adjacent.

- The disk is divided into fixed-size blocks (the smallest unit of data storage).
- The file system stores a file in consecutive (contiguous) blocks.



In the above image on the left side, we have a memory diagram where we can see the blocks of memory. At first, we have a text file named file1.txt which is allocated using contiguous memory allocation, it starts with the memory block 0 and has a length of 4 so it takes the 4 contiguous blocks 0,1,2,3. Similarly, we have an image file and video file named sun.jpg and mov.mp4 respectively, which you can see in the directory that they are stored in the contiguous blocks. 5,6,7 and 9,10,11 respectively.

Here the directory has the entry of each file where it stores the address of the starting block and the required space in terms of the block of memory.

Advantages:

1. Fast Access: Files are read quickly because the blocks are adjacent.
2. Simple to Implement: Only the starting block and file size need to be tracked.

Disadvantages:

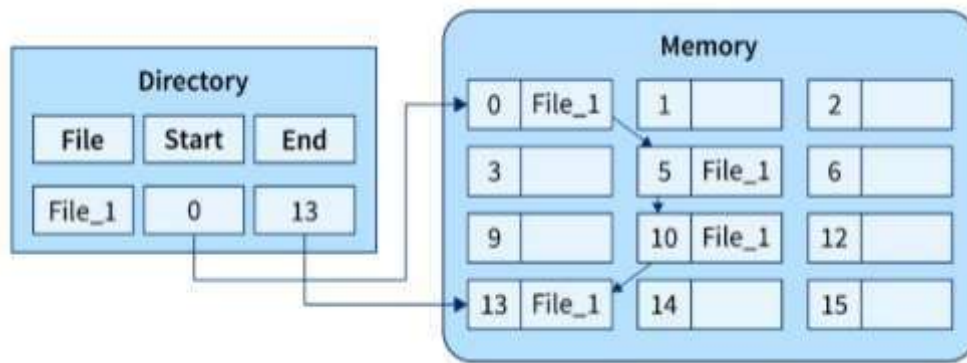
1. External Fragmentation: Gaps form over time, making it hard to find large contiguous spaces.
2. Fixed Size: File size must be known ahead of time and can't easily grow if space is not adjacent.

2) Linked Allocation

Linked allocation is used to overcome the limitations of contiguous allocation. Instead of storing files in adjacent blocks, the file is stored in scattered blocks, with each block pointing to the next block in the file using a link (pointer).

- Each file is stored in a linked list of blocks, meaning each block of the file contains the address (pointer) to the next block of the file.
- This allows files to be scattered throughout the disk rather than needing contiguous space.

- The system stores the pointer to the next block in each block, allowing the file to be located by following these links.
- The last block contains a null pointer (indicating the end of the file).



In the above image on the right, we have a memory diagram where we can see memory blocks. On the left side, we have a directory where we have the information like the address of the first memory block and the last memory block. In this allocation, the starting block given is 0 and the ending block is 15, therefore the OS searches the empty blocks between 0 and 15 and stores the files in available blocks, but along with that it also stores the pointer to the next block in the present block. Hence it requires some extra space to store that link.

Advantages:

1. No External Fragmentation: Files can be stored anywhere, avoiding gaps between blocks.
2. Flexible: Files can easily expand by adding blocks anywhere on the disk.

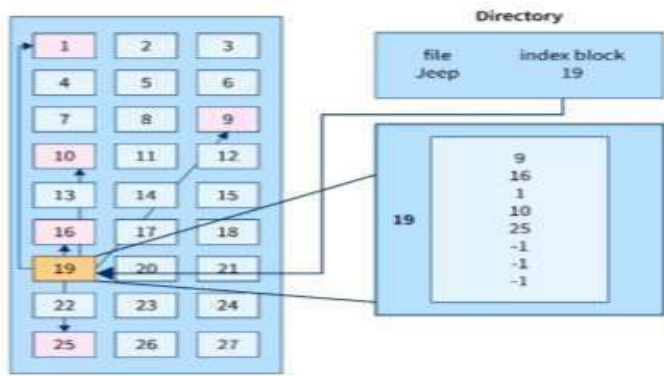
Disadvantages:

1. Slow Access: The system must follow pointers to access each block, making it slower.
2. Extra Storage Overhead: Each block needs space to store a pointer to the next block.

3) Index File Allocation

Indexed allocation combines the benefits of contiguous and linked allocation. In this method, the file system maintains an index block that contains pointers to all the blocks used by a file. The index block allows the system to find all blocks of the file without needing to follow links or search for contiguous space.

- For each file, an index block is created that contains pointers to all the data blocks that the file occupies.
- The index block is stored separately, and it points to each block of the file. The file system can use this index block to quickly access any block of the file directly.
- If the file is large, the index block may itself be split into multiple blocks, creating a multilevel index.



As shown in the diagram below block 19 is the index block which contains all the addresses of the file named text1. In order, the first storage block is 9, followed by 16, 1, then 10, and 25. The negative number -1 here denotes the empty index block list as the file text1 is still too small to fill more blocks.

Advantages:

1. Fast Random Access: The index block allows direct access to any block, speeding up file access.
2. No Fragmentation: Files can be stored anywhere, avoiding external fragmentation.

Disadvantages:

1. Extra Space for Index Blocks: Index blocks take up extra space, and large files may need multiple blocks.
2. Complexity: Managing index blocks increases the complexity of the file system.

Differences

Feature	Contiguous Allocation	Linked Allocation	Indexed Allocation
File Storage	Files are stored in consecutive blocks	Files are stored in scattered blocks	Files are indexed, and data blocks are scattered
Access Speed	Fast (sequential)	Slower (needs to follow pointers)	Fast (random access via index)
External Fragmentation	Yes (can occur over time)	No (uses scattered blocks)	No (uses scattered blocks)
Internal Fragmentation	Low (if the file size is a multiple of block size)	None (no unused space within blocks)	None (depends on index block size)
Overhead	Low (simple management)	High (requires extra space for pointers)	Moderate (requires space for index blocks)

FREE SPACE MANAGEMENT

Free Space Management refers to the process of keeping track of which blocks of disk space are available for use and which are currently allocated to files. The goal is to efficiently manage disk space so that free blocks can be reused for new files or data, avoiding wasted space.

1. **Free Space:** Space on a storage device that is not yet allocated to any file or data.
2. **Free-Space List:** A list or structure maintained by the file system that keeps track of all free (unallocated) blocks on the storage device.
3. **Allocation:** When a new file is created, the system looks for free space from the free-space list and allocates the required number of blocks to the file.
4. **Deallocation:** When a file is deleted, the blocks it occupied are marked as free and added back to the free-space list, making them available for future file allocations.

There are different methods to manage free space.

1) Bit Vector

- The free-space list is implemented as bit vector or bit map.
- Each block is represented by 1 bit.
- If the block is free, the bit is 1; if the block is allocated, the bit is 0.
- For example, consider a disk where blocks 2, 3, 4, 5, 8, 9, 10, 11, 12, 13, 17, 18, 25, 26, and 27 are free and the rest of the blocks are allocated.
- The free space bit map would be 001111001111110001100000011100000

2) Linked List

- Another approach to free-space management is to link together all the free disk blocks, keeping a pointer to the first free block in a special location on the disk and caching it in memory.
- This first block contains a pointer to the next free disk block, and so on.
- Therefore, all the free blocks on the disks will be linked together with a pointer.
- Whenever a block gets allocated, its previous free block will be linked to its next free block.

These methods help the file system manage free space and ensure efficient file creation, deletion, and resizing.

UNIT 3

MEMORY MANAGEMENT

Reference questions

2 Marks Questions:

1. What is the difference between a logical (virtual) address and a physical address?
2. Define dynamic loading and explain one advantage.
3. What is dynamic linking and why is it used in modern operating systems?
4. What is a page fault, and what action does the operating system take when it occurs?
5. Define contiguous memory allocation and explain one of its disadvantages.
6. What is internal fragmentation? Provide an example.
7. What is external fragmentation? How can it be avoided?
8. Explain the term “address translation” in the context of paging.
9. What is the purpose of an inode in file systems?
10. Define a directory in an operating system and name one directory operation.

5 Marks Questions:

1. Explain the process of address translation in a paging system.
2. Describe the swap-in and swap-out operations in virtual memory management.
3. Differentiate between fixed-size and variable-size partitioning in memory management.
4. Describe the concept of pure segmentation and its potential fragmentation issues.
5. What is the role of the Memory Management Unit (MMU) in address translation?
6. Explain the concept of dynamic linking and how it improves memory usage.
7. Describe the structure of a file in an operating system and list its essential attributes.
8. Explain the different types of file allocation methods: contiguous, linked, and indexed allocation.
9. Compare and contrast sequential, direct (random), and indexed-sequential file access methods.
10. What is the purpose of the file system layer structure? Describe its components.

10 Marks Questions:

1. Discuss the differences between virtual and physical memory, explaining how virtual memory allows for a process to exceed physical RAM.
2. Explain how a system handles a page fault and the role of the page replacement algorithm.
3. Compare the different types of memory allocation strategies: contiguous, paging, and segmentation. Discuss the advantages and disadvantages of each.
4. Explain the concept of a file system. Discuss the file allocation methods (contiguous, linked, and indexed), their advantages, and disadvantages.
5. Discuss the file system structure in detail, including the application, logical file system, file-organization module, basic file system, and I/O control layers.
6. Describe the process of dynamic loading and linking. How do they help in reducing memory usage?
7. Explain the role of a directory in a file system and discuss the three common directory structures: single-level, two-level, and hierarchical.
8. What are the steps involved in translating a logical address into a physical address using segmentation and paging?
9. Compare and contrast internal and external fragmentation. Discuss the strategies for handling each type of fragmentation in memory management.
10. Explain the concept of an inode in the context of file systems and list the information it contains. How does the operating system use it to manage files?